



SEMANA 7 · ARQUITECTURAS DE MICROSERVICIOS

Arquitecturas de microservicios

Parte 1 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

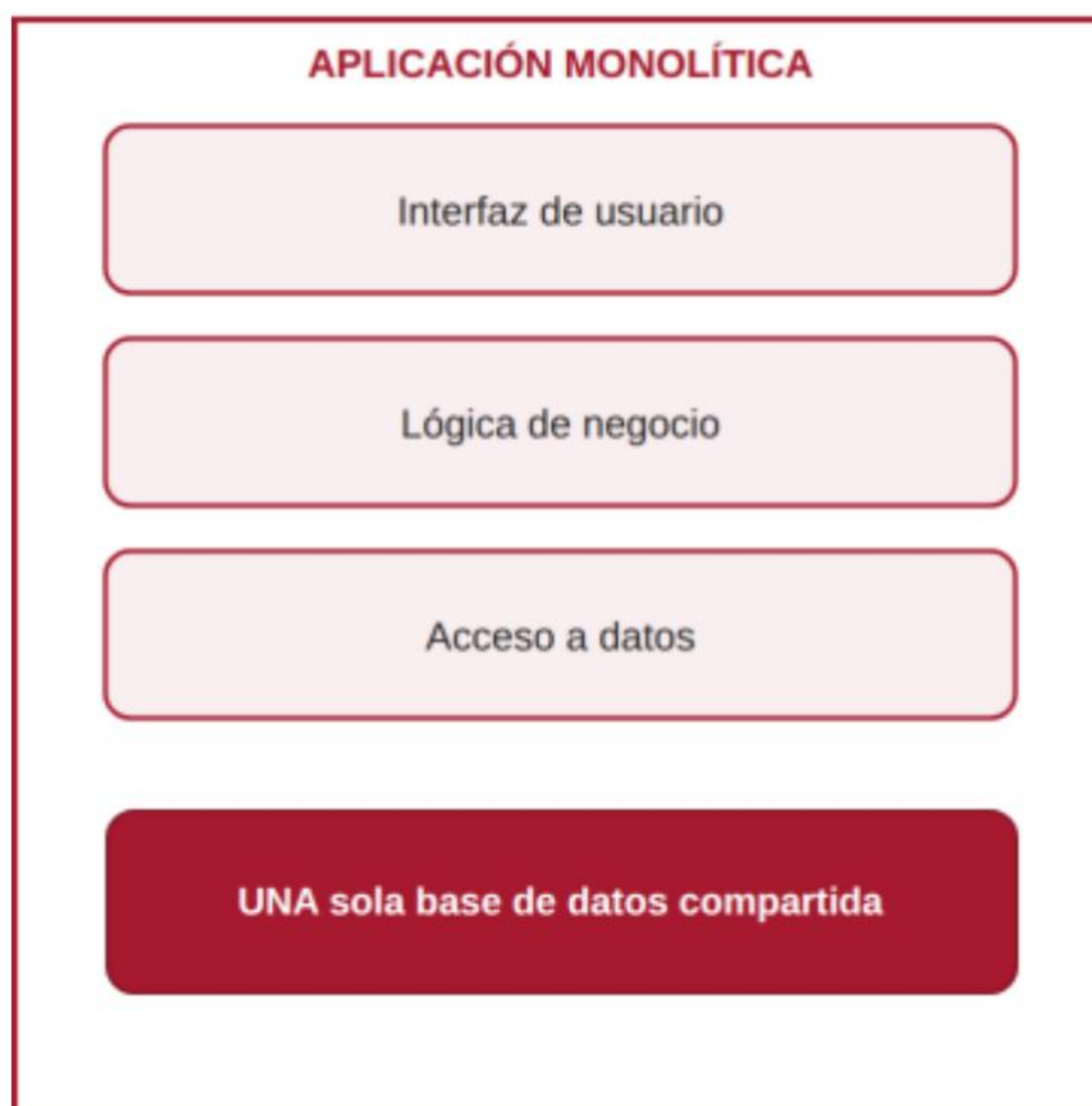
Semana 7 · Periodo 2026-2 · Barranquilla, Colombia

El monolito que creció: diagnóstico del dolor

Monolito y microservicios: dos formas de empacar el sistema

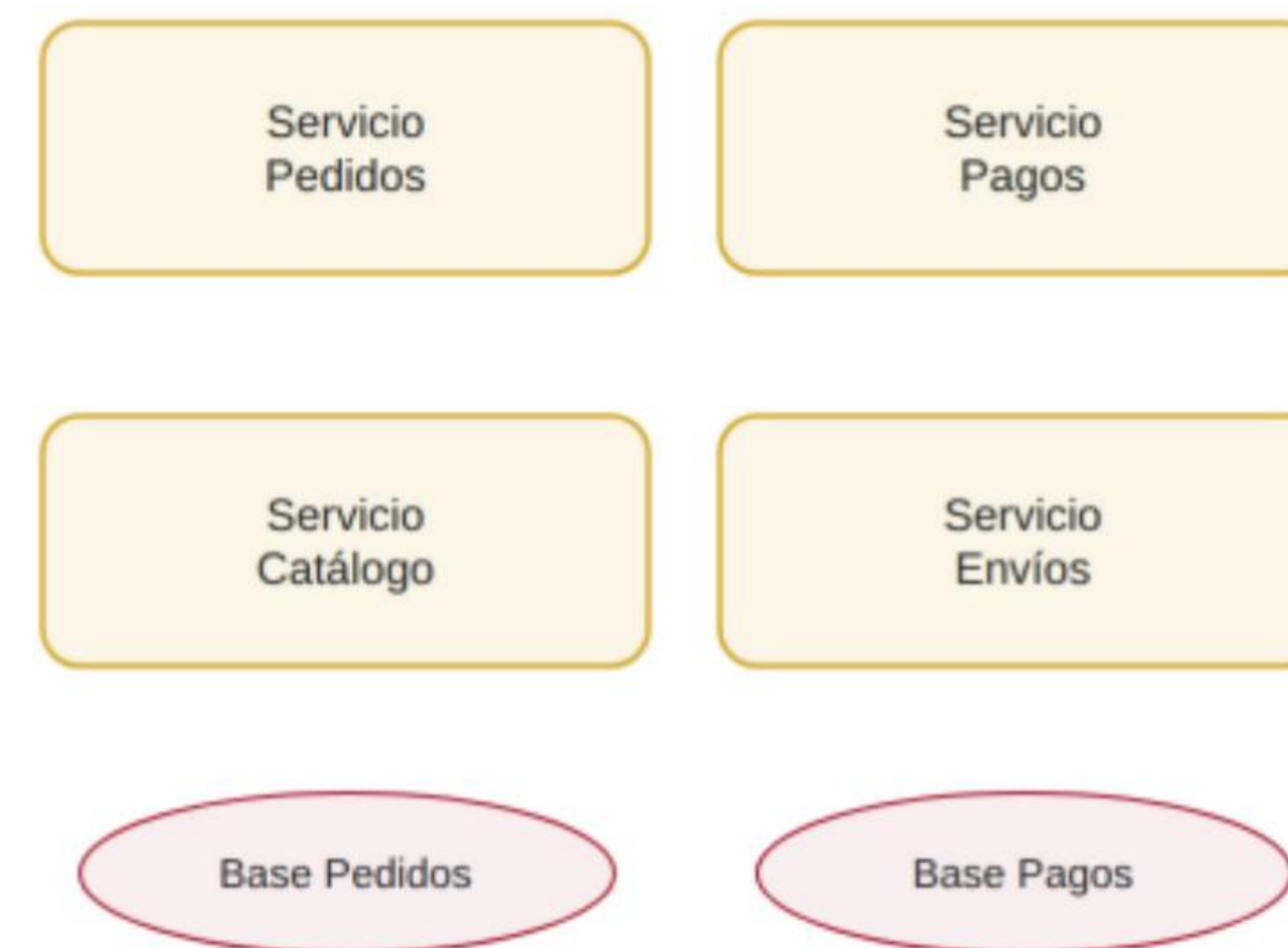
La diferencia no es el tamaño del código, es cómo se despliega y quién es dueño de qué.

MONOLITO: un solo desplegable



un cambio en un módulo obliga a desplegar todo

MICROSERVICIOS: servicios por dominio

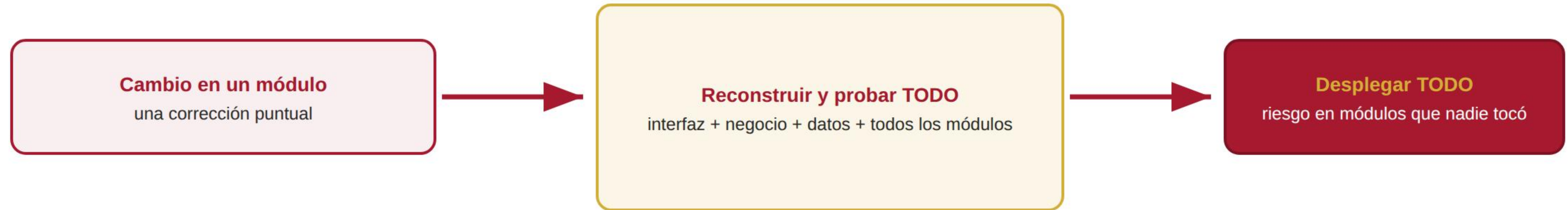


cada servicio es dueño de su dato

La pregunta de diseño no es «¿cuál es mejor?» sino «¿qué me impide hoy cambiar y desplegar solo una parte del sistema?».

El despliegue acoplado: un cambio pequeño mueve todo

Cuando el artefacto es único, cada corrección arrastra el riesgo del sistema completo.



el tamaño del cambio deja de determinar el tamaño del riesgo

señal de diagnóstico: no puedo entregar una parte sin entregar el sistema completo

El fallo en cascada: un módulo tumba el proceso entero

Compartir el proceso significa compartir el destino: un defecto en una función deja sin servicio a todas.



Escalar el todo cuando solo una parte lo necesita

Si no se puede separar, la única palanca es replicar la aplicación completa.

El pico de carga está solo en una función, pero la respuesta es replicar todo

Monolito

instancia 1

Copia completa

instancia 2

Copia completa

instancia 3

Con servicios

solo el servicio con carga crece; los demás quedan igual

se replica lo que no tiene carga

señal: el recurso que se necesita está atado a módulos que no lo necesitan

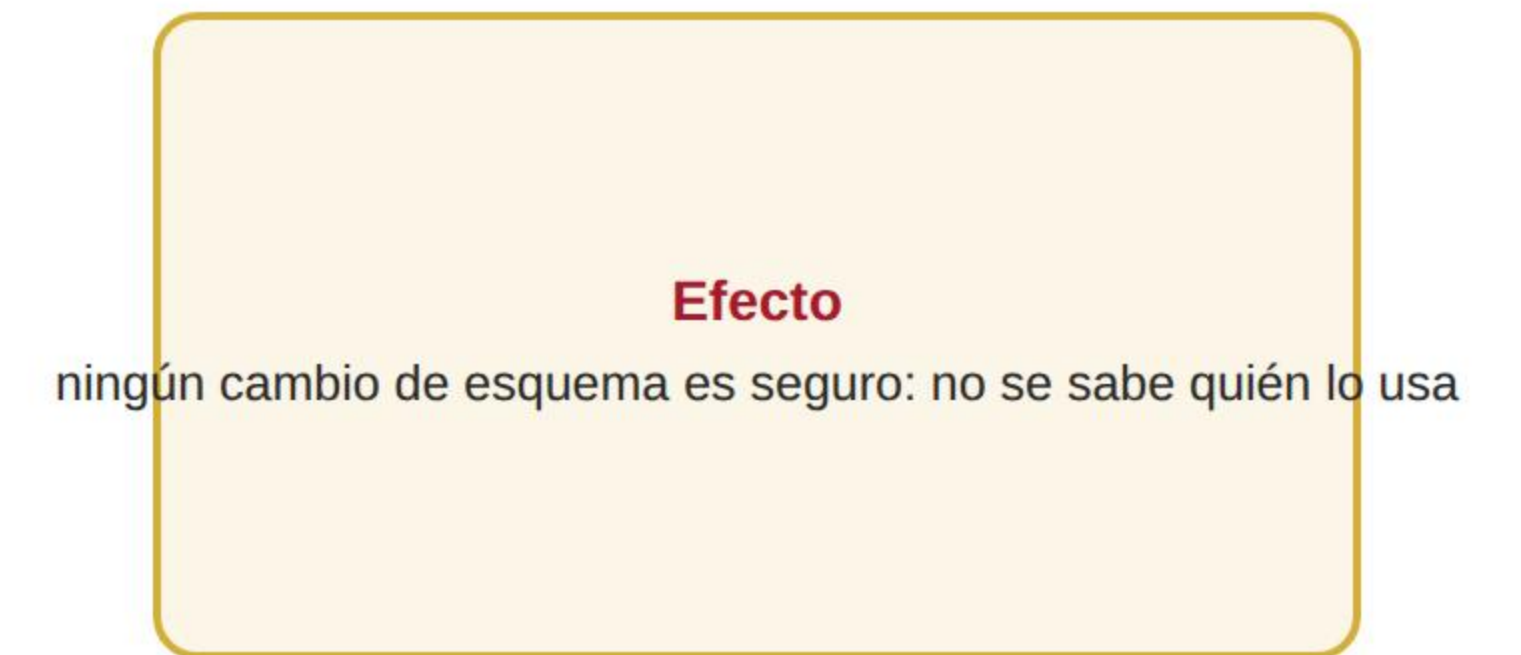
El dato compartido que nadie se atreve a cambiar

Una sola base compartida por todos los módulos convierte cada cambio de esquema en un riesgo global.

Acoplamiento por el dato: varios módulos sobre las mismas tablas



el esquema deja de evolucionar y el sistema se congela



Equipos que se pisan en el mismo artefacto

Un solo artefacto impone una sola línea de entrega para todos los equipos que dependen de él.

Un artefacto, varios equipos

- una sola línea de entrega para todos
- un equipo no publica hasta que otro termina
- la falla de uno detiene la entrega de los demás
- conflictos de integración y colas de coordinación
- el calendario de todos lo decide el más lento

Servicios con dueño

- cada equipo despliega su servicio cuando está listo
- un fallo se limita al servicio que lo introdujo
- las interfaces entre equipos son contratos explícitos
- menos coordinación por cambio
- autonomía real de entrega

Ley de Conway: el diseño del sistema tiende a copiar la estructura de comunicación de la organización. La arquitectura y la organización se condicionan mutuamente. Fuente: Melvin Conway, 1968; Martin Fowler, martinfowler.com.

Una sola versión de todo el stack para todos

Compartir el artefacto obliga a compartir lenguaje, framework y versión de runtime.

UN SOLO RUNTIME PARA TODOS

mismo lenguaje · mismo framework · misma versión

Analítica

necesita una biblioteca no disponible

Seguridad

requiere versión con parches

Frontend

quiere actualizar el framework

Consecuencia

actualizar es una migración global y se posterga

una parte no puede actualizarse sin arrastrar a todas

El monolito no fue un error: fue la decisión correcta a tiempo

Empezar con un solo desplegable suele ser lo apropiado; el problema es no revisar la decisión.

Cuándo el monolito es la elección correcta

- producto nuevo y dominio poco entendido
- equipo pequeño con una sola línea de entrega
- se prioriza velocidad de aprendizaje
- sin necesidad de escalar partes por separado
- una sola base de datos simplifica las transacciones

Cuándo conviene revisar la decisión

- los síntomas de despliegue y fallo ya aparecen
- varios equipos dependen del mismo artefacto
- se necesita escalar una función y no el todo
- la frecuencia de entrega está limitada por el sistema
- el esquema de datos ya no se puede cambiar sin miedo

No es una decisión de una sola vez: la arquitectura se revisa cuando cambian el equipo, la escala o la criticidad — no por moda tecnológica.

Monolito no es sinónimo de desorden

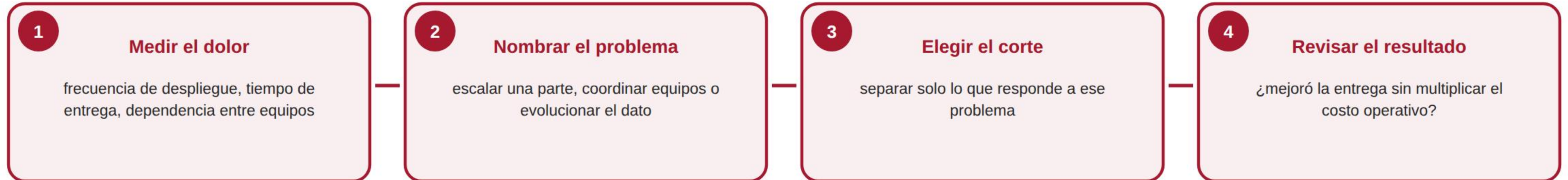
Hay monolitos bien organizados y sistemas distribuidos mal organizados: el despliegue no es el único eje.

Cuatro combinaciones, y solo una es claramente mala



La pregunta correcta no es «¿microservicios sí o no?»

Descomponer es una respuesta a un problema medible, no una meta de arquitectura.



Criterio: si no hay un problema medible que la descomposición resuelva, la complejidad distribuida se paga sin obtener beneficio.

Descomponer temprano también se paga

La complejidad distribuida tiene un precio en operación, diagnóstico y datos.

Lo que se gana

- despliegue independiente por servicio
- límites de fallo: un servicio no tumba el resto
- escalado selectivo de la parte que lo necesita
- dueño claro del dato y de la interfaz
- autonomía de equipos y de tecnología

Lo que cuesta

- más piezas que operar, monitorear y asegurar
- diagnóstico distribuido: hace falta trazabilidad
- consistencia de datos entre servicios (sagas)
- latencia de red y llamadas que pueden fallar
- gobierno de contratos entre servicios

Regla: la descomposición se justifica cuando el beneficio (autonomía, escalado, evolución del dato) supera el costo operativo que introduce.

Un mapa de dolor para decidir

Cada síntoma tiene una señal observable y una acción correspondiente — no todas exigen microservicios.

Síntoma	Señal observable	Respuesta posible
Despliegue acoplado	el tiempo de entrega crece	separar la capacidad que más cambia
Fallo en cascada	incidentes afectan funciones no relacionadas	aislar el proceso de la parte crítica
Escalar el todo	capacidad ociosa y costo alto	separar el servicio con la carga
Dato compartido	el esquema no se puede cambiar	definir propiedad del dato
Equipos bloqueados	colas de coordinación	límites con dueño explícito
Tecnología congelada	actualizaciones postergadas	aislar el componente a actualizar

Lo que queda de la Parte 1

El monolito no se juzga por su forma, sino por lo que impide hacer.

6

síntomas observables del monolito que
creció

1

artefacto desplegable y una base
compartida en el monolito

0

servicios que deberían compartir la
misma base de datos

4

costos que introduce la descomposición:
operación, diagnóstico, consistencia,
latencia

Siguiente pregunta: si hay que cortar, ¿dónde se corta? ¿Por capa técnica —interfaz, negocio, datos— o por dominio del negocio?



SEMANA 7 · ARQUITECTURAS DE MICROSERVICIOS

Arquitecturas de microservicios

Parte 2 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 7 · Periodo 2026-2 · Barranquilla, Colombia

Descomposición por dominio: dónde cortar

El corte correcto es por dominio, no por capa técnica

Separar interfaz, negocio y datos produce tres equipos que siempre cambian juntos.

CORTE POR CAPA TÉCNICA



CORTE POR DOMINIO



Qué es un contexto delimitado

Un límite donde un modelo y su lenguaje tienen un significado único y consistente.

Un dominio grande se divide en contextos delimitados

CONTEXTO VENTAS

«cliente» = quien compra
datos de contacto y crédito
su propio modelo y su base

CONTEXTO SOPORTE

«cliente» = quien reporta
historial de casos y SLA
su propio modelo y su base

CONTEXTO FACTURACIÓN

«cliente» = a quién se factura
datos fiscales y de pago
su propio modelo y su base

el mismo término, tres significados: esa divergencia es la señal del límite

El mismo nombre con significados distintos

Cuando dos áreas definen igual un término, puede que en realidad deban ser parte del mismo contexto.

«Producto» según cada contexto

CATÁLOGO

descripción, imágenes, precio,
disponibilidad para vender

qué muestra al cliente

INVENTARIO

referencia, ubicaciones,
unidades y lotes

dónde está físicamente

FACTURACIÓN

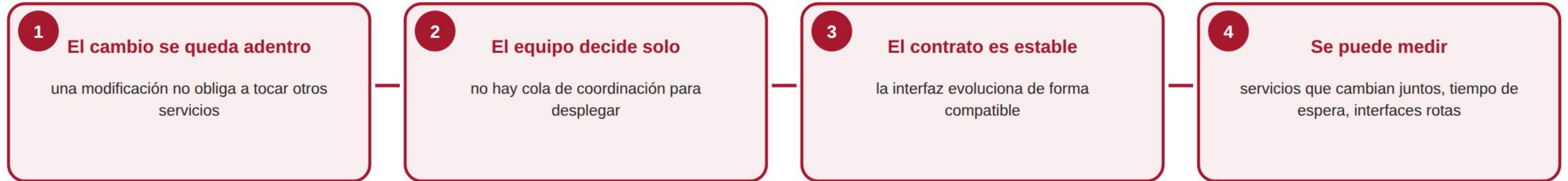
descripción fiscal,
impuesto y total

qué se cobra

tres modelos distintos del mismo término: forzarlos a una tabla produce el dato compartido

Cómo se reconoce un buen corte

El cambio se queda dentro del servicio, el equipo decide solo y el contrato es estable.



Indicador de éxito: después del corte, ¿cuántos servicios cambian juntos en un mismo cambio? Si el número no baja, el límite quedó mal trazado.

Cómo se reconoce un mal corte

Llamadas constantes, transacciones que abarcan servicios y datos que todos leen: señales inequívocas.

Señales de que el límite está mal trazado

Llamada constante entre servicios

cada petición exige consultar al otro: la información necesaria vive al otro lado

Transacción que cruza el límite

la operación exige escribir en dos servicios de forma atómica

Dato que todos leen

nadie confía en la copia del otro y todos consultan la misma fuente

Servicio sin lógica propia

existe solo para exponer datos de otro: frontera artificial

La interfaz entre servicios es un contrato explícito

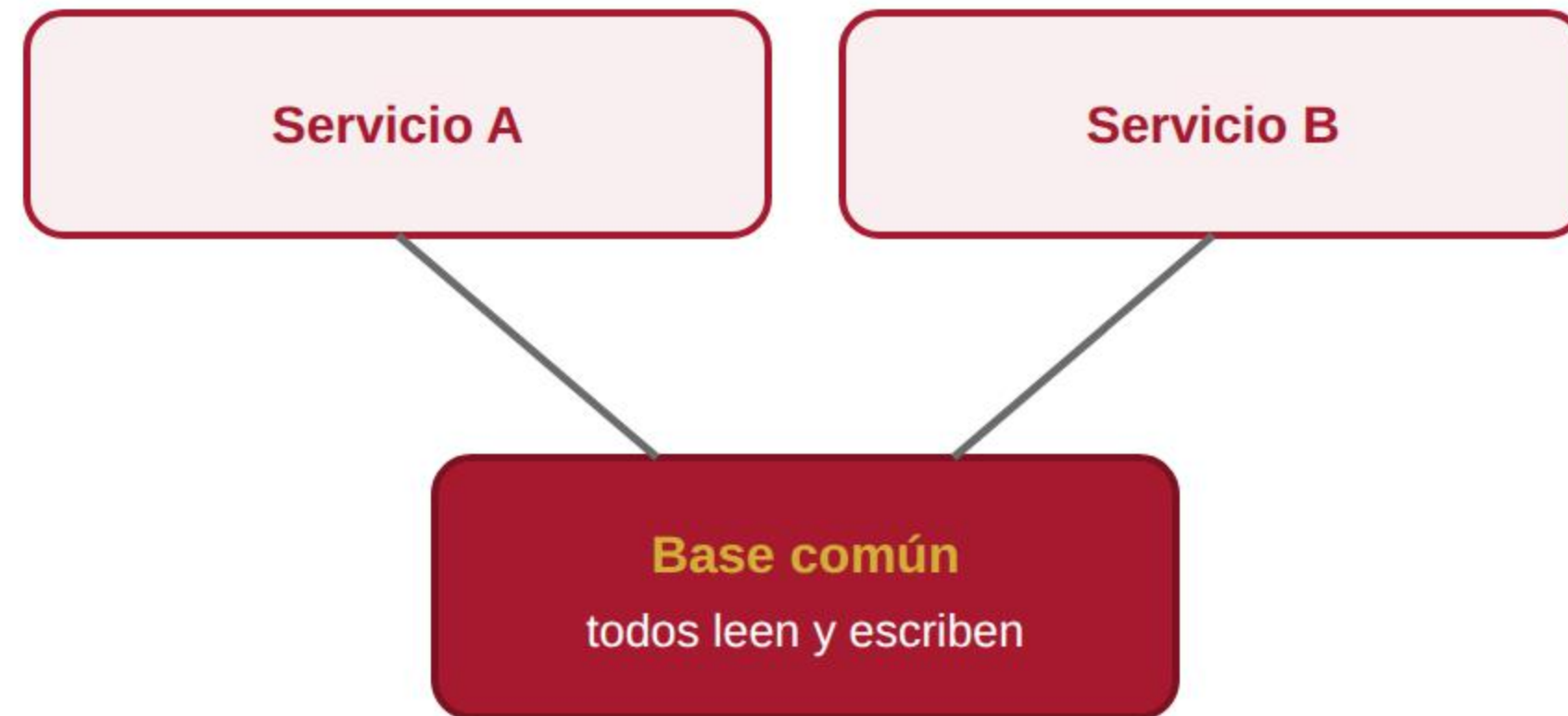
Lo que se comparte no es el código ni la base: es un acuerdo de comunicación versionado.



Una base por servicio: quién es dueño del dato

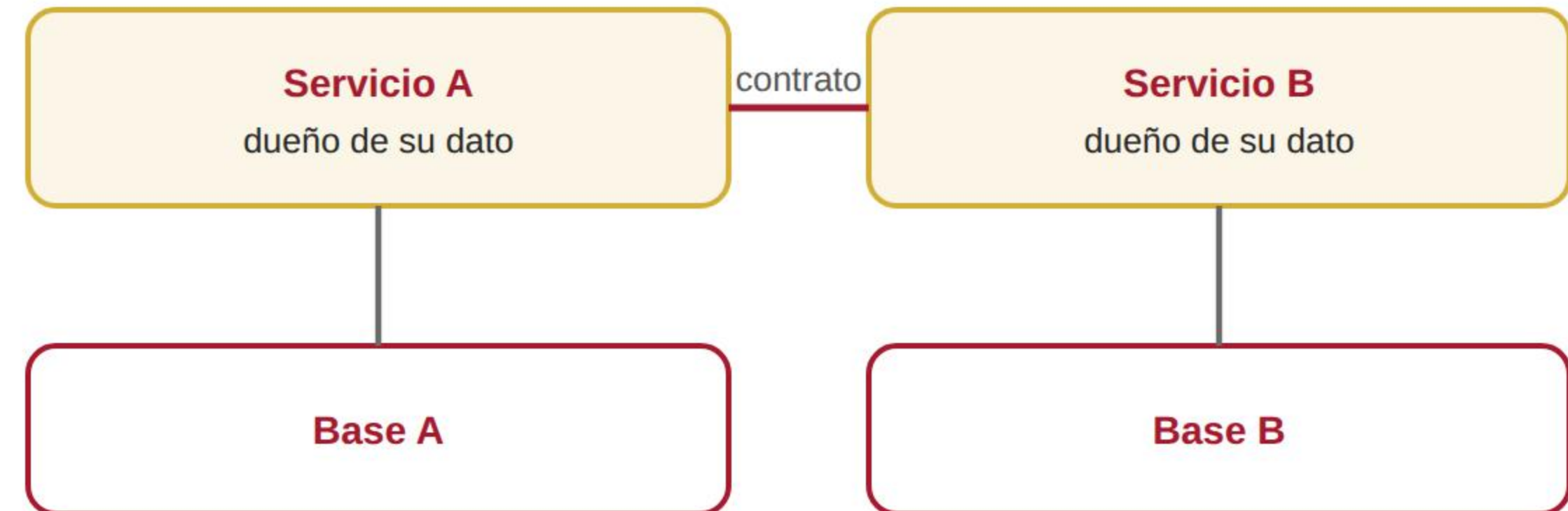
La regla que hace real la separación: solo el dueño escribe; nadie más lee su base directamente.

Base compartida: separación aparente



un cambio de esquema rompe a todos

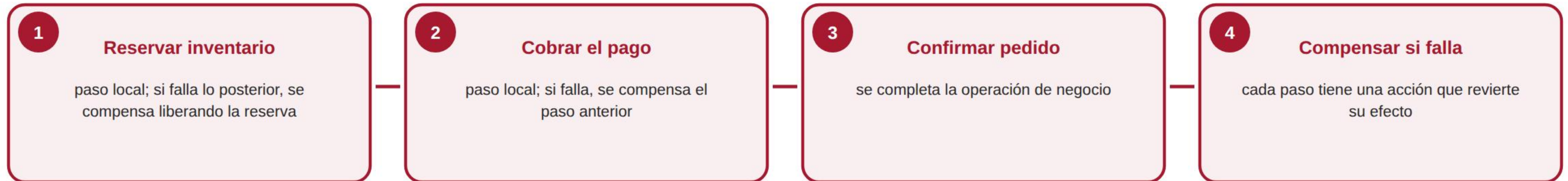
Base por servicio: separación real



cada uno cambia su modelo sin romper al otro

De la transacción única a la saga con compensación

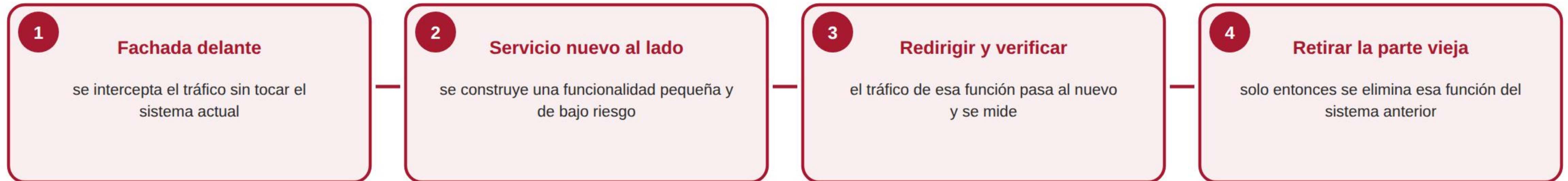
Una operación que toca dos servicios ya no es atómica: se coordina con pasos y acciones que revierten.



Consistencia eventual: entre un paso y su compensación el sistema está en un estado intermedio — el negocio debe poder tolerarlo, y eso se decide en el diseño.

El patrón del estrangulador: descomponer sin reescribir todo

Se construye lo nuevo alrededor de lo viejo y se redirige el tráfico por partes, sin apagar el sistema.



Ventaja: cada paso es pequeño y reversible. Si algo falla, se devuelve el tráfico al sistema viejo sin apagar la operación.

¿Qué tan pequeño es un microservicio?

«Micro» describe el alcance de la responsabilidad, no el número de líneas.

Demasiado fragmentado

- servicios que siempre cambian juntos
- cada operación cruza varios servicios
- llamadas constantes entre límites artificiales
- demasiadas piezas que operar sin autonomía real

Tamaño razonable

- dueño completo de una capacidad de negocio
- puede entregar valor sin coordinar
- sus cambios se quedan dentro del servicio
- su interfaz es lo único que otros necesitan conocer

Regla práctica: la unidad correcta no es «lo más pequeño posible», es un contexto delimitado con responsabilidad completa y despliegue independiente.

El límite y el equipo: Conway en la práctica

Un servicio que nadie mantiene no es un servicio, es un problema futuro.

Cada servicio necesita un dueño identificable en el tiempo

Equipo A

dueño del servicio de pedidos y del de catálogo

Equipo B

dueño del servicio de pagos

Equipo C

dueño del servicio de envíos

Responsabilidad

decidir sobre tecnología y despliegue dentro de un marco común · responder por la disponibilidad y la evolución

no hace falta un equipo por servicio; sí hace falta un dueño claro por servicio

Monolito distribuido: partir sin cambiar el acoplamiento

Separar el despliegue y dejar intactas las dependencias produce lo peor de ambos mundos.



se paga la complejidad de lo distribuido sin obtener autonomía

Lo que queda de la Parte 2

Se corta por dominio, se descubre el contexto, se es dueño del dato y se respeta el contrato.

2

errores de corte: por capa técnica y
monolito distribuido

1

dueño exclusivo del dato por servicio:
solo él lee y escribe su base

0

servicios que deberían compartir la
misma base de datos

3

condiciones de un buen corte: cambio
local, equipo autónomo, contrato
estable

Siguiente pregunta: cuando dos servicios necesitan hablar, ¿el emisor espera la respuesta o puede continuar sin esperarla?



SEMANA 7 · ARQUITECTURAS DE MICROSERVICIOS

Arquitecturas de microservicios

Parte 3 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

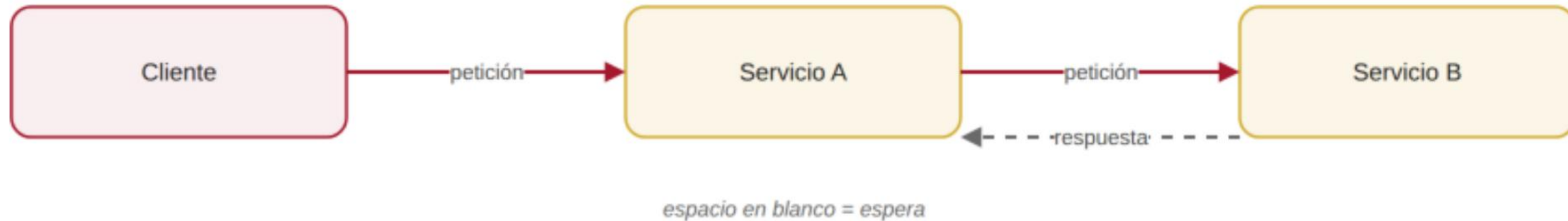
Semana 7 · Periodo 2026-2 · Barranquilla, Colombia

Comunicación síncrona y asíncrona entre servicios

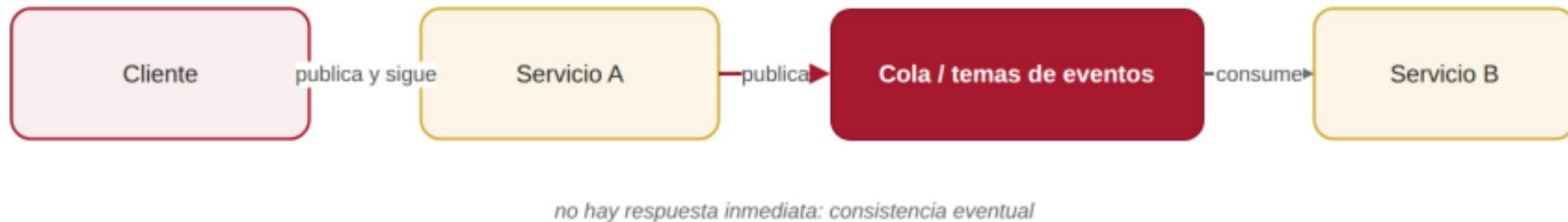
Dos formas de comunicar servicios, y una decisión de arquitectura

Esperar la respuesta o continuar sin esperarla cambia el acoplamiento del sistema.

SÍNCRONO: el emisor espera la respuesta



ASÍNCRONO: el emisor continúa sin esperar

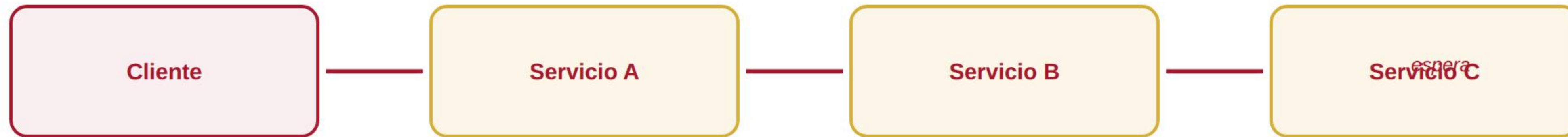


La pregunta que decide: ¿el emisor puede continuar con su trabajo sin conocer la respuesta inmediata? Si puede, la comunicación asíncrona es candidata.

Síncrono: el emisor espera y la cadena crece

Cada petición encadena tiempos de espera, y ese encadenamiento acumula latencia.

La latencia se acumula a lo largo de la cadena de esperas



latencia total = espera A + espera B + espera C + red

y el cliente permanece bloqueado todo ese tiempo

El costo real de lo síncrono: acoplamiento temporal

Si el servicio que responde no está disponible, el emisor tampoco puede completar su trabajo.

Un eslabón caído detiene toda la cadena



la disponibilidad de la cadena es el producto de sus eslabones, no el promedio

y las fallas en cadena generan tormentas de reintentos que agravan la carga

Cuatro defensas para lo síncrono

Si no se puede dejar de esperar, al menos hay que limitar el daño de esa espera.

Tiempo límite

ninguna llamada espera indefinidamente: una espera sin límite consume recursos y arrastra fallos

Reintentos con espera progresiva

se repite una llamada transitoria, aumentando el intervalo para no castigar al servicio

Cortocircuito

tras varios fallos se deja de llamar por un tiempo y se devuelve una respuesta controlada

Degradación planificada

si un auxiliar no responde, la operación principal continúa en versión reducida

no eliminan el acontecimiento temporal; impiden que un fallo local se vuelva una caída total

Asíncrono: el emisor publica y continúa

El productor no espera; el consumidor procesa cuando puede y el sistema gana elasticidad.



amortiguación de carga: los picos se acumulan y se procesan a capacidad estable

si el consumidor cae, los mensajes quedan guardados hasta que vuelva

Cola de tareas o publicación de eventos

No es lo mismo pedir que alguien haga algo que anunciar que algo ocurrió.

Cola de tareas

- el productor publica una orden concreta
- un mensaje es procesado por un consumidor
- el productor espera que se haga el trabajo
- útil para repartir trabajos una sola vez
- ejemplo: generar un documento, enviar un correo

Publicación de eventos

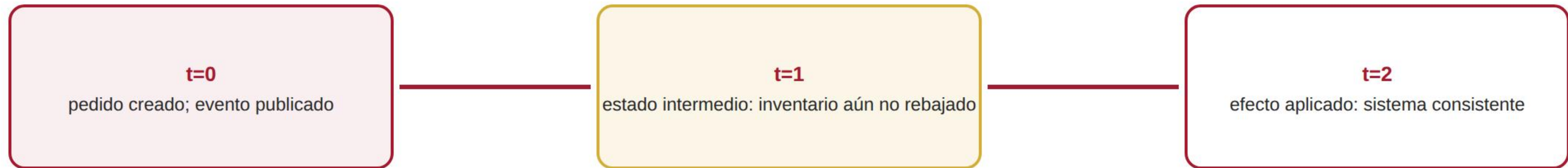
- el productor anuncia un hecho ocurrido
- no se dirige a nadie en particular
- varios consumidores se suscriben por su cuenta
- el consumidor decide qué hacer con el evento
- ejemplo: pedido creado, pago confirmado

Diferencia clave: en la tarea el productor ordena; en el evento informa y los demás reaccionan. Los sistemas maduros combinan ambos modelos.

El costo real de lo asíncrono: consistencia eventual

Sin respuesta inmediata, el sistema pasa por estados intermedios que el negocio debe tolerar.

Estado del sistema entre la publicación y el procesamiento



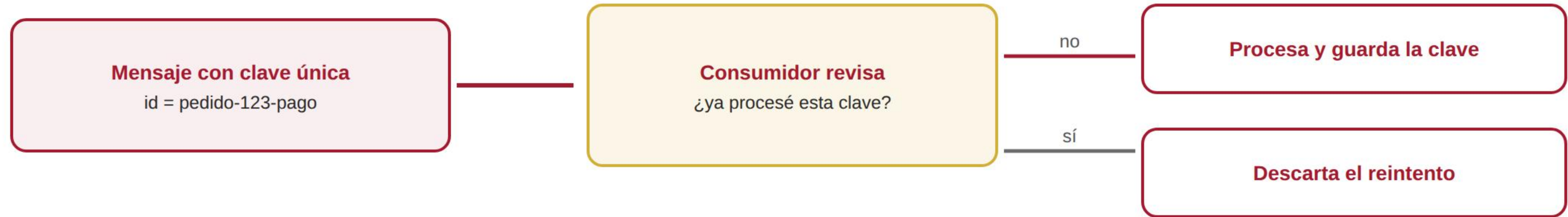
consistencia eventual: no es inmediata, pero llega

la decisión es por operación: se acepta donde el negocio tolera el intervalo

Idempotencia: el concepto que evita duplicados

En un sistema que reintenta, un mensaje puede procesarse dos veces; hay que diseñarlo para que no importe.

Una clave de idempotencia convierte un reintento en una repetición sin efecto



idempotente: aplicarla varias veces da el mismo resultado que aplicarla una vez

para las operaciones que no lo son, la clave de idempotencia lo resuelve

El orden de los mensajes no está garantizado

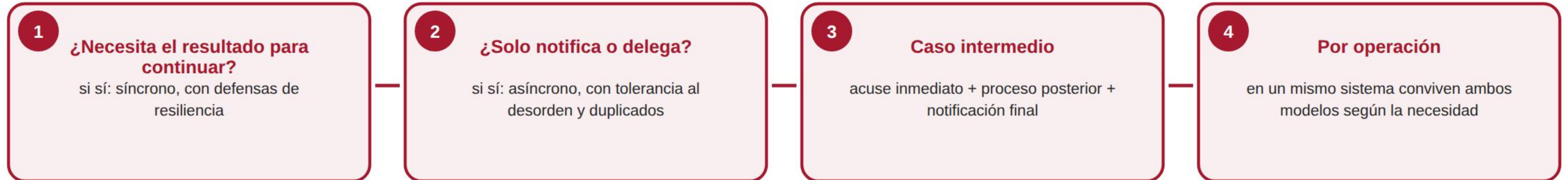
Diseñar asumiendo desorden evita errores que solo aparecen bajo carga.

El orden de llegada no es el orden de envío



Cómo elegir entre síncrono y asíncrono

La pregunta es operativa: ¿el emisor necesita el resultado para continuar con su trabajo?



No es una decisión única: consultas suelen ser síncronas; cambios de estado y notificaciones, asíncronas. Se elige operación por operación.

Coreografía y orquestación

Un proceso que cruza varios servicios se coordina con eventos o con un director explícito.

Coreografía

- cada servicio reacciona a eventos y publica los suyos
- no hay director central
- descentralizado y flexible
- difícil de seguir cuando hay muchos pasos
- el recorrido está repartido entre todos

Orquestación

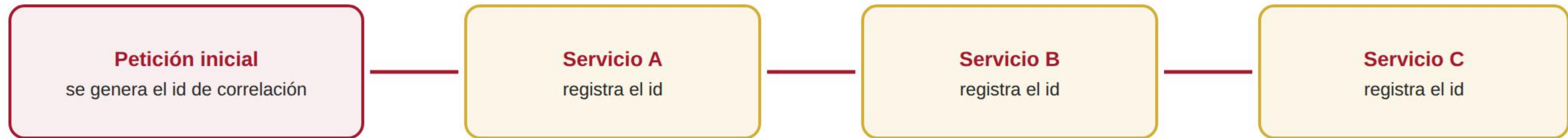
- un coordinador dirige la secuencia de pasos
- el estado del proceso vive en un solo lugar
- más fácil de visualizar, depurar y auditar
- introduce un componente central
- útil en procesos largos con compensaciones

Criterio práctico: si nadie puede responder con precisión en qué paso va una operación, el proceso pide orquestación; si los pasos son pocos y reactivos, la coreografía es suficiente.

Sin trazabilidad, el fallo distribuido es invisible

Un identificador que viaja con la petición es lo que permite reconstruir el recorrido completo.

El identificador de correlación acompaña toda la operación



con el id completo se reconstruye el recorrido y se ubica el punto de fallo

métricas por servicio: latencia, tasa de error y saturación; alertas antes de afectar al usuario

Lo que queda de la Parte 3

Síncrono cuando hay que esperar; asíncrono cuando se puede continuar; idempotencia y trazas siempre.

2

modelos de comunicación: síncrono (espera) y asíncrono (continúa)

2

modelos asíncronos: cola de tareas y publicación de eventos

4

defensas de lo síncrono: tiempo límite, reintento, cortocircuito, degradación

0

operaciones que deberían asumir orden y unicidad de los mensajes

Siguiente pregunta: si los servicios ya están separados y se comunican, ¿cómo llega el cliente externo sin conocer ni exponer cada servicio?



SEMANA 7 · ARQUITECTURAS DE MICROSERVICIOS

Arquitecturas de microservicios

Parte 4 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

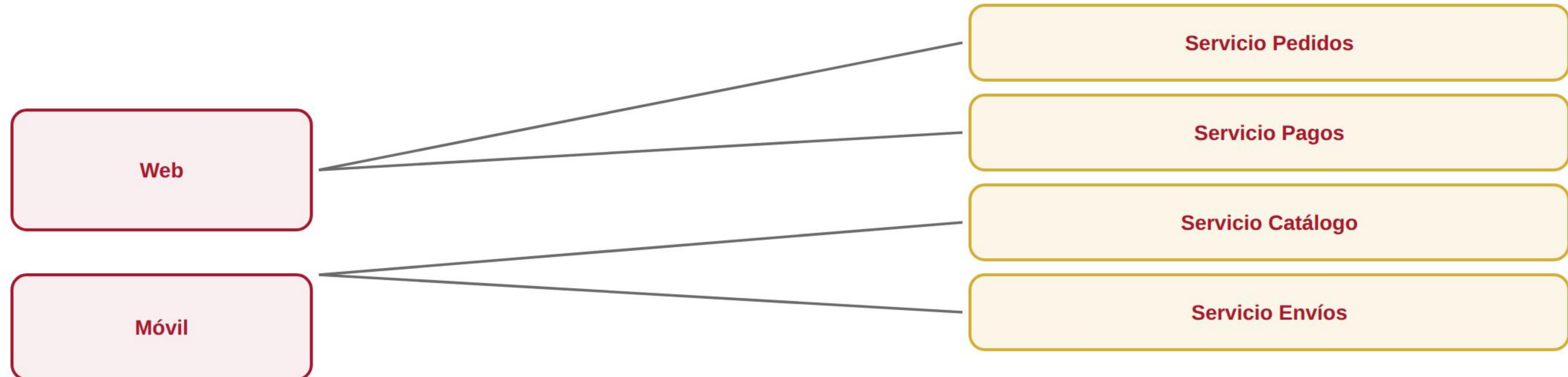
Semana 7 · Periodo 2026-2 · Barranquilla, Colombia

API Gateway y despliegue de los servicios

Por qué no se expone cada servicio directamente

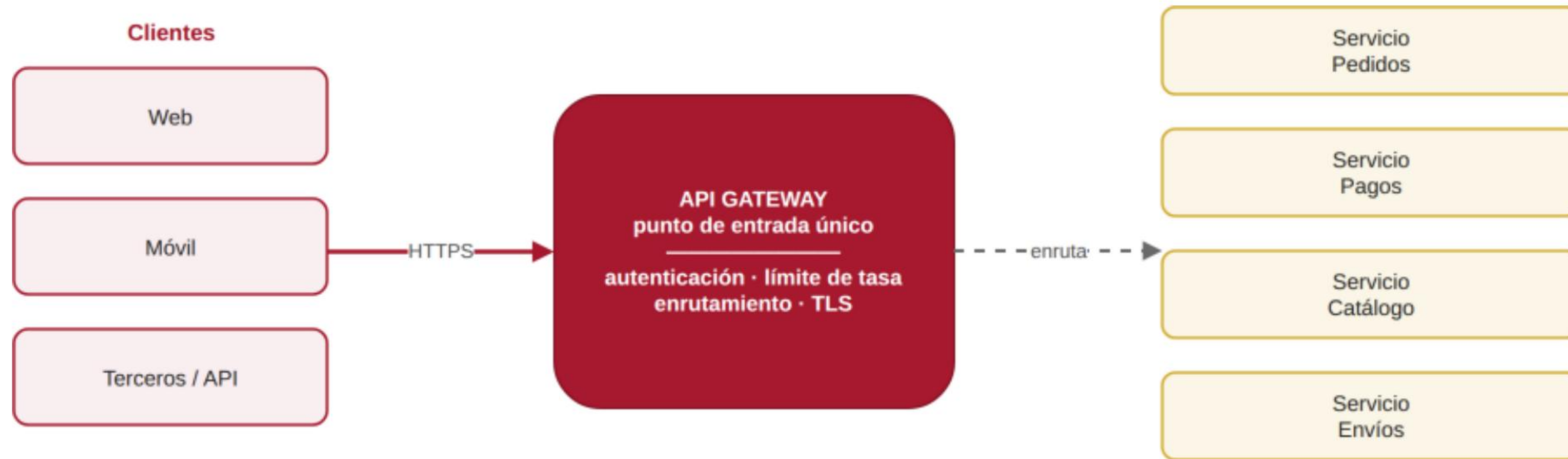
Si el cliente conoce todos los servicios, cada cambio de descomposición se convierte en una actualización de los clientes.

Exposición directa: el cliente conoce y llama a cada servicio



El API Gateway: un punto de entrada único

El cliente habla con una sola dirección; el enrutamiento y las políticas viven detrás.



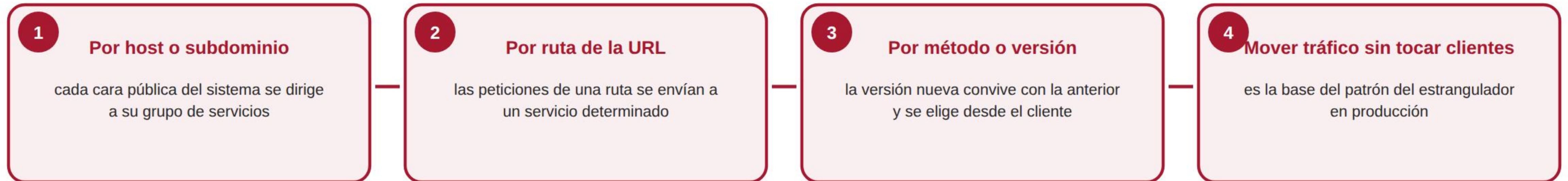
los servicios internos nunca se exponen directamente a internet

el gateway no debe contener la lógica de negocio

Regla de oro: los servicios internos nunca se exponen directamente a internet; la puerta de enlace es la única cara pública del sistema.

Enrutamiento: cada petición encuentra su servicio

La ruta se decide por host, por ruta de la URL, por método o por versión.



Habilitador de la migración: mover tráfico por reglas permite redirigir funciones al sistema nuevo, verificar y luego retirar la parte vieja.

Autenticación centralizada y propagación de identidad

La puerta valida una vez y transmite la identidad; los servicios no repiten la validación de credenciales.

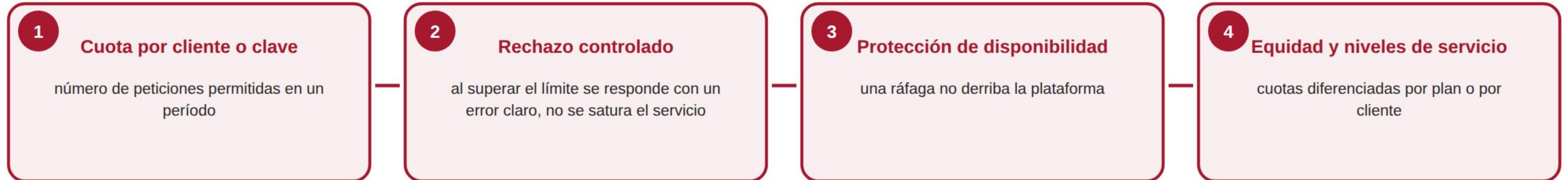


la puerta no reemplaza el mínimo privilegio dentro de cada servicio

la red interna está aislada: solo la puerta recibe tráfico externo

Límite de tasa: proteger los servicios de los picos

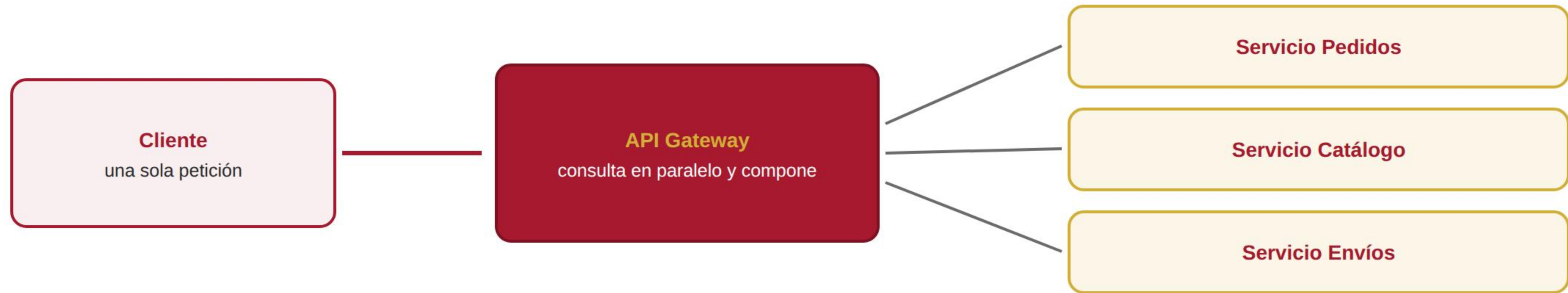
La puerta impone cuotas por cliente y evita que una ráfaga derribe la plataforma.



Decisión de arquitectura: los límites se fijan según la capacidad real de los servicios y los acuerdos de nivel de servicio, y se comunican a los consumidores.

Composición de respuestas: una llamada, varios servicios

La puerta puede combinar datos de varios servicios y devolver al cliente una sola respuesta coherente.



componer respuestas sí; aplicar reglas de negocio no

si la puerta decide precios o descuentos, concentra la complejidad que queríamos distribuir

Quando los clientes necesitan cosas distintas

Una puerta por tipo de cliente evita que cada uno reciba datos de más.

Una sola puerta para todos

- menos capas que mantener
- una respuesta común para todos los canales
- el móvil recibe campos que no usa
- más datos transferidos de los necesarios
- suficiente si los clientes son parecidos

Una capa por tipo de cliente

- cada cliente recibe lo que necesita
- respuestas adaptadas por canal
- los servicios internos no cambian
- más capas que evolucionar y mantener
- se justifica si los canales son muy distintos y estables

Patrón backend para frontends: una capa de composición por tipo de cliente. Se justifica cuando las diferencias son reales y estables, no por una variación menor.

El gateway no es el lugar de la lógica de negocio

Cuando la puerta decide reglas del dominio, se convierte en un cuello de botella y en un riesgo de cambio.

Cómo se degrada una puerta de enlace



El punto de entrada único también es un punto único de fallo

Si toda la plataforma pasa por la puerta, la puerta tiene que ser redundante y observada.



Consecuencia del punto único: la disponibilidad del sistema completo queda limitada por la de la puerta. La redundancia y el monitoreo no son opcionales.

Qué debe hacer la puerta por la seguridad del borde

Terminación de cifrado, inspección básica y cero exposición de servicios internos.



Cómo se despliegan estos servicios: puente a la semana 8

Una imagen por servicio, un registro de imágenes y un despliegue que no depende del servidor.

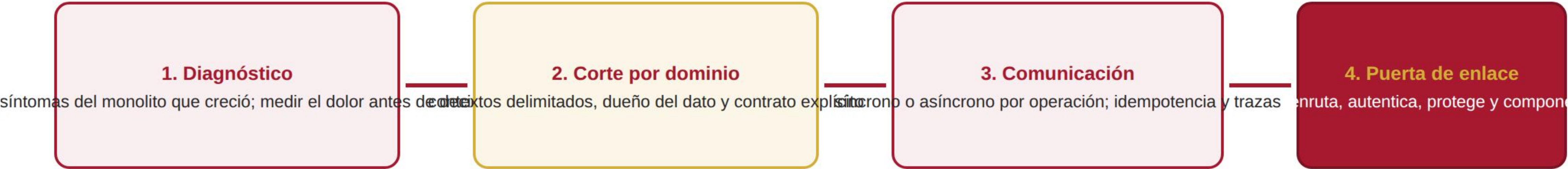


Puente a la semana 8: contenedores, imágenes y registro (ECR) — con el laboratorio del 5%: publicar una imagen en el registro y ejecutarla.

El recorrido completo de la semana en un solo diagrama

Del diagnóstico del monolito a la puerta de enlace, pasando por el dominio y la comunicación.

Semana 7: de los síntomas al punto de entrada único



y el despliegue de todo esto: contenedores e imágenes versionadas (semana 8)

una arquitectura de servicios es un conjunto de decisiones, no un producto que se instala

Lo que queda de la Parte 4 y de la semana

Un punto de entrada único, con límites claros, y el puente hacia el despliegue con contenedores.

4

responsabilidades de la puerta:
enrutar, autenticar, proteger, componer

4

problemas que resuelve el punto de
entrada único

0

reglas de negocio que deben vivir en la
puerta de enlace

5%

laboratorio evaluado de la semana 8:
imagen en el registro (ECR)

Semana 8: contenedores e imágenes; construir una imagen, publicarla versionada en el registro y ejecutarla. Es el laboratorio del 5% — conviene llegar con el diseño de esta semana claro.